ファームウェアにおけるアセンブリ命令列と制御フローの差分を用いた脆弱性修正の特定手法

西岡 謙介[1,a)]　奥川 莞多[2,1]　毛利 公一[1]

概要：IoT 機器ではファームウェアに含まれるソフトウェアの脆弱性を悪用した攻撃が報告されており，更新によって脆弱性が適切に修正されているか確認する必要がある．一般的な確認手段として SCA ツールが用いられるが，メタデータに依存するため，バージョン情報を変更しない暗黙的パッチやベンダ独自の修正の検出には限界がある．この課題を解決するため，実行ファイルそのものを比較するバイナリ差分解析が用いられる．しかし，脆弱性修正を確認する既存のバイナリ差分解析手法は，特定の脆弱性に対する修正パターンとの照合を前提としているため，修正内容自体が未知である更新の中から脆弱性修正を特定・抽出することは困難である．そこで本論文では，修正内容の詳細が不明なファームウェア更新から脆弱性修正を特定・抽出するため，更新前後でファームウェアに含まれる ELF ファイルを比較し，アセンブリ命令列および制御フローの変化を抽出した上で，これらの変化を既存研究で整理されたソースコード上の脆弱性修正の特徴と対応付ける手法を提案する．

キーワード：バイナリ解析，脆弱性修正，ファームウェア

Identifying Vulnerability Fixes by Analyzing Differences in Assembly Instruction Sequences and Control Flow Between Firmware Versions

Kensuke Nishioka[1,a)]　Kanta Okugawa[2,1]　Koichi Mouri[1]

Abstract: Attacks exploiting software vulnerabilities in the firmware of IoT devices have been reported, making it necessary to verify whether these vulnerabilities have been properly fixed through updates. While SCA tools are commonly used for this verification, they rely on metadata and therefore have limitations in detecting implicit patches that do not change version information, as well as vendor-specific fixes. To address this issue, binary difference analysis, which compares the executable files themselves, is employed. However, existing binary difference analysis methods for verifying vulnerability fixes are based on matching against known fix patterns for specific vulnerabilities; therefore, it is difficult to identify and extract vulnerability fixes from updates where the details of the fixes themselves are unknown. In this paper, to identify and extract vulnerability fixes from firmware updates where the details of the fixes are unknown, we propose a method that compares the ELF files contained in the firmware before and after the update, extracts changes in assembly instruction sequences and control flow, and then maps these changes to the characteristics of vulnerability fixes in source code as categorized in existing research.

Keywords: Binary Analysis, Vulnerability Fixes, Firmware

1. はじめに

IoT 機器のファームウェアに含まれるソフトウェアの脆弱性を悪用した攻撃が報告されている [1]．このような脆弱性への対策として，ベンダはファームウェア更新を提供す

[1] 立命館大学，Ritsumeikan University
[2] 国立研究開発法人情報通信研究機構，National Institute of Information and Communications Technology
[a)] knishioka@asl.cs.ritsumei.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

る．Dong ら [2] は，OSS のバージョン情報に基づく調査により，N-day 脆弱性の影響を受けると判定されたソフトウェアや，リリース時点で既知脆弱性を含む古いバージョンのソフトウェアがファームウェアに組み込まれていることを報告している．ただし，ファームウェアには，修正を適用してもバージョン情報を変更しない暗黙的パッチが存在する [3]．このため，ファームウェア更新による脆弱性修正を確認する上では，更新によってバイナリに現れた変化を捉えることが重要となる．

バイナリを用いた脆弱性修正の調査方法は，修正内容に関する事前情報の有無によって異なる．修正対象となる脆弱性とその修正パッチが公開されている場合には，修正パッチから生成した特徴を対象バイナリと照合し，パッチの適用有無を判定する手法が提案されている [4]．これに対し，修正対象や具体的な修正内容が公開されていないファームウェア更新では，修正パッチに基づく照合用の特徴を事前に用意できない．そのため，更新前後のバイナリを直接比較し，更新によって生じた差分から脆弱性修正に対応する箇所を探索する必要がある．しかし，この差分には，脆弱性修正だけでなく，リファクタリングなど他の変更も混在する．したがって，修正内容に関する事前情報を利用できないファームウェア更新では，バイナリ差分から脆弱性修正に対応する変更を切り分けることが必要となる．

バイナリ差分に混在する変更を切り分ける手掛かりとして，ソースコード差分に現れる脆弱性修正の特徴を利用することが考えられる．先行研究 [5] では，構文および依存関係の変化に基づいて，複数の変更が混在するソースコード差分から脆弱性修正を切り分ける手法が提案されている．しかし，IoT 機器では解析対象のソースコードを入手できず，バイナリのみが提供される場合が多いため，ソースコード上の特徴をそのまま利用することはできない．

そこで，本論文では，先行研究 [5] で整理されたソースコード上の脆弱性修正の特徴を，アセンブリ命令，基本ブロック，制御フローおよび関数呼出しに現れる変化として捉え直す．この対応付けに基づいて更新前後のファームウェアに含まれる ELF バイナリを比較し，複数の変更が混在するバイナリ差分から脆弱性修正候補を抽出する手法を提案する．提案手法の評価では，脆弱性修正前後のソースコードと修正履歴を利用できる OSS を対象とし，ソースコード上の修正を正解データとして，対応するバイナリ差分を抽出できるかを検証した．その結果，評価対象とした 6 つの外部公開関数すべてにおいて，CVE に対応する修正を含む基本ブロックを脆弱性修正候補として抽出できた．また，CVE に直接対応しない入力値検査についても，脆弱性修正と共通する特徴を持つ変更として抽出可能であった．以上の結果から，ソースコード上の特徴をバイナリ上の変化へ対応付けることにより，バイナリ差分から脆弱性修正候補を絞り込めることを示した．

2. 背景

2.1 バージョン情報に基づく脆弱性修正状況の調査

ファームウェアに含まれる OSS と既知脆弱性との対応を調査する手段として，Software Composition Analysis（SCA）が用いられる．SCA では，ファイル名や実行ファイル中の文字列などからソフトウェア名とバージョンを推定し，脆弱性データベースと照合する．この方法は，ファームウェアに含まれるソフトウェアと既知脆弱性との関係を広範に調査する上で有効である．一方，IoT 機器では，必要な修正のみを旧バージョンへバックポートするケースやベンダ独自の修正が適用される場合がある．こうした修正では，プログラムが変更されても，バージョン情報が更新されないことがある．このように，バージョン情報を変更せずに適用される修正は，暗黙的パッチと呼ばれ，SCA を用いたファームウェアの脆弱性修正状況を正確に把握することを阻害している．秋山らによる調査 [3] では，バージョン番号が同一である時系列上のプログラムに生じた変更の約 44.3%が暗黙的パッチであると推定しており，バージョン情報に基づく判定だけでは，修正済みのソフトウェアを脆弱と判定する可能性があると指摘している．

2.2 ソースコード差分に基づく脆弱性修正の分析

脆弱性修正を調査する際，対象のソースコードを利用できる場合には，更新前後のソースコード差分から変更された処理を直接分析できる．脆弱性修正に着目してソースコード差分を調査した研究として，Li ら [6] は，682 の OSS リポジトリから収集した 4,080 件の脆弱性修正を，一般的な不具合修正と比較している．その結果，脆弱性修正における変更行数の中央値は 7 行であり，一般的な不具合修正における 16 行よりも少なかった．また，脆弱性修正の 59%は単一の関数内に収まっており，脆弱性修正が比較的小規模かつ局所的な変更として現れる傾向を示している．

Sun ら [5] は，NVD および PatchDB から収集した 5,000 件のセキュリティパッチを人手で分析し，GitHub から無作為に抽出した 5,000 件の非セキュリティコミットと比較することで，12 種類の脆弱性修正パターンと 3 種類の非セキュリティ変更パターンを整理している．Sun らが提案した DISPATCH は，これらのパターンに加えて，更新前後における構文，制御依存，データ依存および関数呼出し関係の変化を分析する．これにより，複数の目的を含むソースコード差分を個別の変更単位へ分離し，脆弱性修正に対応する変更を切り分ける．このように，ソースコードを利用できる場合には，構文および依存関係に関する情報を手掛かりとして，差分に混在する変更の目的を分析できる．

2.3 バイナリ差分に基づく脆弱性修正の分析

IoT 機器のファームウェアに対応するソースコードを利

© 2026 Information Processing Society of Japan

用できない場合には，更新前後の実行ファイルを直接比較するバイナリ差分解析が用いられる．BinDiff[7] や Ghidra Version Tracking[8] などの差分解析ツールは，二つのバイナリ間で関数や基本ブロックを対応付け，アセンブリ命令列および制御フローグラフの変化を抽出する．ファームウェアへの適用例として，兼松ら [9] は，更新前後のファームウェアに含まれる実行ファイルを比較し，差分が生じた関数を特定することで，バージョン情報を変更せずに加えられた脆弱性修正の確認を支援している．

このようなバイナリ差分解析により変更箇所を特定できる一方，その変更が脆弱性修正，通常の不具合修正または機能変更のいずれに対応するかまでは判定できない．そのため，抽出された差分から脆弱性修正を特定するには，差分の内容から変更目的を識別する必要がある．

バイナリ差分から変更目的を識別する手法は，対象となる修正に関する事前情報の有無によって二つに整理できる．一つは，既知の修正パッチから生成した特徴を対象バイナリと照合する手法である [4], [10], [11], [12]．この問題は一般に Patch Presence Test と呼ばれる．FIBER[4] は，対象となる修正パッチと影響を受ける関数が既知であることを前提として，パッチに対応する局所的な制御フローおよび意味的特徴を抽出し，対象バイナリへの適用有無を判定する．もう一つは，具体的な修正内容を事前に与えず，更新前後のバイナリ差分からセキュリティに関連する変更を識別する手法である．SPAIN[13] は，更新によって変化した実行トレースを意味的に分析し，脆弱性修正と非セキュリティ変更を識別する．BinGo[14] は，変更された基本ブロックの命令と，制御フロー，制御依存およびデータ依存を統合したグラフを用いて，バイナリ差分を脆弱性修正または非セキュリティ変更へ分類する．

2.4 ファームウェア更新全体の分析における課題

既存研究を整理すると，SCA は OSS と既知脆弱性との関係を調査できる一方，バージョン情報だけでは暗黙的パッチの適用有無を判定できない．ソースコード差分を用いる手法は解析対象のソースコードを必要とし，Patch Presence Test は対象となる修正パッチを必要とする．ソースコードや修正パッチを利用できない場合には，更新前後のバイナリ差分が修正内容を調査する手掛かりとなるが，バイナリ差分解析ツールは変更された関数や基本ブロックを示すものであり，その変更目的までは判定しない．

この問題に対し，修正パッチを事前に与えることなく，バイナリ差分を脆弱性修正または非セキュリティ変更に分類する手法が提案されている．しかし，これらの手法は，あらかじめ一つの修正単位として切り出された差分を分析対象としているため，脆弱性修正，不具合修正，機能変更などが混在し得るファームウェア更新における各 ELF バイナリの差分を分類することはできない．したがって，ソースコードや対象固有の修正パッチを利用せず，ファームウェアに含まれる ELF バイナリの差分から，脆弱性修正に関連する関数および基本ブロックを候補として抽出することが課題となる．

3. バイナリ差分に基づく脆弱性修正候補の抽出手法

本章では，更新前後のファームウェアに含まれる ELF バイナリを比較し，その差分から脆弱性修正候補を抽出する手法について説明する．バイナリ差分解析ツールは変更箇所を特定できる一方，その変更目的までは判断できない．先行研究の DISPATCH[5] は，12 種類の脆弱性修正パターンと，ソースコード上の Anchor Node を整理している．そこで，本研究では，これらの修正パターンと Anchor Node をもとに，バイナリ上で観測可能な命令や基本ブロックを Anchor Node として再定義し，いずれかの修正パターンに一致する差分を脆弱性修正候補として抽出する．

3.1 提案手法の処理フロー

提案手法の全体像を図 1 に示す．本手法は，事前準備，差分箇所の特定とコードの対応付け，および脆弱性修正候補の分類の 3 段階から構成される．なお，本手法では，32 ビット ARM アーキテクチャ向けの ELF バイナリを解析対象としている．

事前準備では，更新前後の ELF バイナリを Ghidra へ入力し，BinExport プラグインを用いて BinDiff の入力ファイルを生成する．

差分箇所の特定とコードの対応付けでは，生成した BinExport ファイルを拡張した BinDiff へ入力し，更新前後で対応する関数および基本ブロックを特定する．拡張した BinDiff から，差分基本ブロックの開始アドレスと終了アドレス，命令または基本ブロックの追加，削除，更新の種別，ならびに制御フローグラフの変化を差分情報として取得する．これと並行して，更新前後のバイナリを `objdump` で逆アセンブルする．その後，BinDiff から得られたアドレス情報を用いて逆アセンブル結果を参照し，各差分基本ブロックと，その内部に含まれるアセンブリ命令列とを対応付ける．最終的には，構造的な差分情報である基本ブロックと，命令単位の差分情報であるアセンブリ命令を統合したバイナリ差分情報を生成する．

脆弱性修正候補の分類では，まず，差分基本ブロックの命令列から，判定の起点となる Anchor Node を探索する．次に，Anchor Node 周辺の命令列，制御フロー，および関数呼出し関係を解析し，バイナリ上で観測可能な特徴として再定義した 12 種類の脆弱性修正パターンとの対応を判定する．いずれかのパターンに該当する差分を脆弱性修正候補に分類し，それ以外をその他の変更に分類する．

© 2026 Information Processing Society of Japan

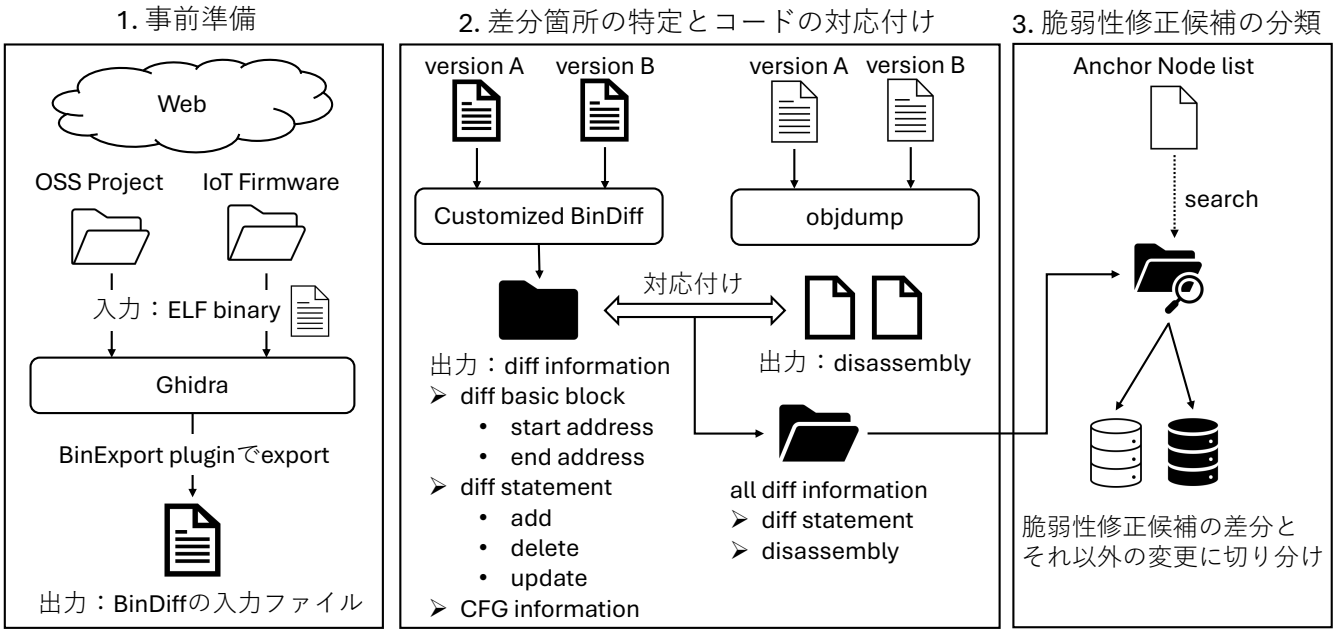


図 1 バイナリ差分を用いた脆弱性修正候補を抽出する提案手法の流れ

3.2 脆弱性修正パターンとバイナリの Anchor Node

DISPATCH[5] が整理した 12 種類の脆弱性修正パターンについて，ソースコードにおける検出対象を説明し，バイナリ上で Anchor Node として着目する候補を対応付ける．

Sanity Check は，異常な入力値や戻り値を検査して，異常時の処理を追加または強化するパターンである．例えば，`if (p == NULL) return -1;` のような条件検査の追加は Add Security Check に対応する．また，`if (n < 0)` を `if (n <= 0)` へ変更するような検査条件の厳格化は Update Security Check に対応する．

Function は，資源管理または排他制御を行う関数呼出しに着目する．例えば，`free(p)` の追加，`free` をメモリ内容を消去してから解放する関数に置き換える変更，lock/unlock の追加・移動，および複数箇所の `if` 文を新たな関数としてカプセル化する変更が含まれる．

Variable は，`char *p;` を `char *p = NULL;` へ変更するような初期化，または `char buf[128]` を `char buf[256]` へ変更するようなバッファの増加を対象とする．

Statement は，エラー状態を返す `return` 文の変更，複数の `return` を `goto err;` へ置換して共通の後処理へ移す変更，および `void` 型関数から `int` 型関数などへの変更を対象とする．

これらの変更をバイナリ上で捉えるための代表的な Anchor 候補を表 1 に示す．バイナリにおける Anchor Node は，コンパイルにより一つ以上の命令または基本ブロックとして現れる．本手法では，対応する差分命令または差分基本ブロックを Anchor 候補として定義している．

表 1 修正パターンに対する代表的なバイナリ上の Anchor 候補

パターン	Anchor 候補
Sanity Check	
Add/Update Security Check	条件付き分岐，`CBZ`/`CBNZ`，A32 の条件実行命令，または T32 の IT block
Function	
Add Mem Deallocation Func	メモリ解放 API の呼出し
Update Mem Management Func	メモリ管理 API の呼出し
Add Concurrency Control	lock 取得 API の呼出し
Update Concurrency Control	lock 取得 API の呼出し
Function Encapsulation	条件付き分岐とヘルパ関数の呼出し
Variable	
Initialize Variable	ゼロ値生成命令と格納命令，または `memset` の呼出し
Enlarge Buffer Size	スタック確保命令，malloc の引数設定命令
Statement	
Update Return Value	関数復帰直前における `r0` の最終定義の変更
Enhance Error Handling	エラー処理または共通終了処理への無条件分岐
Update Return Type	関数復帰直前に新たに追加された `r0` の定義

3.3 Anchor Node を利用した脆弱性修正候補の抽出

提案手法では，定義した Anchor Node に加えて，BinDiff が出力した基本ブロックの差分情報と基本ブロック内の命令がどのように変更されたかという情報を組み合わせて，

© 2026 Information Processing Society of Japan

各脆弱性修正候補を抽出する．以降，各カテゴリの検出方法について述べる．

Sanity Check カテゴリの検出： ソースコード上の `if` 文は，バイナリ上で比較・フラグ設定命令と条件付き分岐または CBZ/CBNZ，条件実行命令，IT block として現れる．そこで，`ADDED` または `UPDATED` の条件付き分岐を Anchor 候補として検出し，直前の比較・フラグ設定命令を補助条件として用いる．対象とする命令形には，比較と条件分岐の組に加え，`CBZ/CBNZ`，A32 の条件実行命令，および T32 の IT block を含める．

Function カテゴリの検出： 想定するファームウェアでは通常のシンボルテーブルが削除されている一方で，動的リンクされる共有ライブラリの関数名は動的シンボル表に残る場合がある．そこで，呼出し先を解決できる直接呼出しや外部 API への直接末尾呼出しの命令である `BL`，`BLX`，`B/B.W` を Anchor 候補として，`objdump` の出力から取得した呼出し先シンボルを API リストと照合する．API リストは，C 標準ライブラリや POSIX API として提供される API 名を対象とする．メモリ管理では `free`，`cfree` などのメモリ解放 API を，排他制御では `pthread_mutex_lock`，`pthread_spin_lock`，`sem_wait` などの lock 取得 API を検出する．Function Encapsulation では，削除された条件分岐と追加されたヘルパ関数への呼出しを，基本ブロック対応および関数呼出し関係から関連付けて検出する．

Variable カテゴリの検出： Initialize Variable では，`sp` または `fp` を基準とするスタックへの格納命令を特定し，同一基本ブロック内でその格納値を生成する命令を確認する．具体的には，ゼロ値生成命令が新たに導入され，その値が当該スタックへ格納される場合を初期化候補とする．また，第 2 引数に 0 が設定された `memset` への直接呼出しが追加された場合も候補として検出する．Enlarge Buffer Size では，バイナリ上で観測可能な確保容量の増加を，ローカル領域とヒープ領域に分けて検出する．ローカル領域では，関数入口のスタック確保命令に指定された即値で比較し，スタック確保量が増加した場合を候補とする．ヒープ領域では，メモリ確保 API への直接呼出しを特定し，ARM 32-bit ABI に基づいて確保容量を表す定数引数を比較し，容量が増加した場合を候補とする．提案手法ではコード領域の差分を対象とするため，`.data` や `.bss` などのデータ領域にのみ現れる変更は検出できない．

Statement カテゴリの検出： ARM 32-bit ABI では返り値が `r0` に格納されるため，Update Return Value および Update Return Type では，return 命令で終わる基本ブロックにおける `r0` への最終定義を Anchor 候補とする．Update Return Value では，修正前後で `r0` に格納される値が変更されたか，Update Return Type では，`r0` に格納される値が修正後のみ追加された状態を検出する．Enhance Error Handling では，共通 return 命令で終わる基本ブロックへ分岐する無条件分岐を Anchor 候補とし，修正前のリターン経路および修正後の分岐先を制御フローグラフから確認する．

いずれかの脆弱性修正パターンに一致した場合，Anchor ID，関数名，基本ブロック番号，修正に該当するアドレス範囲，一致した命令列，各命令の差分状態，および呼出し先・制御フローなどの根拠を脆弱性修正候補として出力する．いずれの Anchor 候補にも一致しない差分は未分類差分として保存する．

4. 実験

本章では，提案手法が実際のソフトウェアに含まれる既知の脆弱性修正を捉えられるかを確認するために実施した実験について述べる．

4.1 実験目的

CVE に対応する修正が行われた外部公開関数を正解データとし，提案手法がその脆弱性修正の変更をバイナリ差分から検出できるか確認する．さらに，同じ外部公開関数に CVE 関連の修正以外の変更が行われている場合，その内容をソースコードから確認し，提案手法が変更の特徴を捉えられるかを調査する．以上の目的に基づき，次の検証項目を設定する．

- 検証項目 1：バージョンアップ全体としてどれほど脆弱性修正の特徴を示す変更が行われているか
- 検証項目 2：提案手法は，外部公開関数に行われた CVE 関連の脆弱性修正をバイナリから検出できるか
- 検証項目 3：同一の外部公開関数において，CVE に関連しない変更も含めてどのような脆弱性修正候補を検出するか

なお，脆弱性に関する変更には，CVE として公開された修正だけでなく，CVE が割り当てられていない不具合修正，予防的なセキュリティ強化，および非公開の脆弱性修正が含まれる可能性がある．このため，CVE に対応しない検出結果を誤検出とはみなさず，ソースコードの変更内容を確認した上で評価する．

4.2 実験対象

実験対象には，XML パーサライブラリである Expat を用いた．Expat は，GitHub 上でソースコードと CVE に対応する修正履歴が公開されており，ソースコード上の変更を正解データとして利用できるため対象とした．また，共有ライブラリでは外部公開関数のシンボルが残るため，strip されたバイナリであってもソースコード上の関数と対応付けることができる．比較対象には，Debian において armhf（32 ビット ARM）向けに配布されている libexpat1 パッケージのバージョン 2.8.1-1 および 2.8.2-1 を用いた．パッケージ版と Upstream 版の対応を表 2 に示す．また，

© 2026 Information Processing Society of Japan

表 2　実験対象とした Expat ライブラリのバージョン

Debian パッケージ名	Debian パッケージ版	Upstream 版
libexpat1_2.8.1-1	2.8.1-1	2.8.1
libexpat1_2.8.2-1	2.8.2-1	2.8.2

両 Debian パッケージに含まれるパッチファイルのハッシュ値を比較した結果，同名のパッチに差異がないことを確認した．したがって，本実験で観測されるバイナリ差分は，Upstream 版の Expat 2.8.1 から 2.8.2 への更新のみであるため，該当の GitHub リポジトリの差分を用いて変更内容を分類する．

4.3　実験方法

Expat の GitHub リポジトリからバージョン 2.8.1，2.8.2 に対応するソースコードを取得し，`git diff` を用いてバージョン間の変更内容を調査する．Expat では，CVE が割り当てられた脆弱性修正と対応する Pull Request（PR）が公開されているため，バージョンアップで修正された 10 件の CVE に関して PR 単位で修正内容を確認し，各変更をソースコードにおける脆弱性修正パターンに分類する．

次に，CVE 関連の修正が行われた関数のうち，バイナリにシンボル名が残る外部公開関数を抽出した．その結果，3 件の CVE に対応する 6 関数を正解データとした．内部関数はシンボル名が削除され，ソースコードとの対応付けが困難であるため，正解データには含めていない．

最後に，提案手法で二つの共有ライブラリから差分基本ブロックと脆弱性修正パターンを取得し，正解データとの比較によって CVE 関連の修正を検出できたかを確認するとともに，同じ外部公開関数に含まれる CVE 以外の変更についてもソースコードと照合した．

4.4　実験結果

検証項目 1：バージョンアップ全体の検出結果

提案手法によって検出されたバージョン間の差分基本ブロックの集計結果を表 3 に示す．バージョン間では 1,725 個の差分基本ブロックが検出され，このうち 694 個がいずれかの脆弱性修正パターンに一致した．11 個の基本ブロックは複数のパターンに一致したため，表 3 の合計は延べ 705 個となる．提案手法で脆弱性修正候補として検出された基本ブロックのほとんどは，Security Check の追加または更新であった．

検証項目 2：CVE 関連の脆弱性修正の検出結果

正解データとした外部公開関数ごとの検出結果を表 4 に示す．表中の CVE 対応のソースコード分類は，対応する PR のソースコード差分を手作業で分類した結果である．正解データとした 6 関数すべてで，CVE 修正に対応する変更を含む基本ブロックが少なくとも一つ検出された．手作業でバイナリの差分をソースコード差分と照合したところ，提案手法ではバイナリ差分から CVE 関連の脆弱性修正候補を関数単位で漏れなく抽出できていた．

表 3　提案手法による脆弱性修正パターンの検出結果

脆弱性修正パターン	基本ブロック数
Add Security Check	404
Update Security Check	295
Update Return Value	1
Initialize Variable	3
Enlarge Buffer Size	2
合計（重複も含む）	705

代表例として，`XML_Parse` における PR#1246 の修正を示す．

```
+   if (isCalledFromInsideHandler(parser))
+       return XML_STATUS_ERROR;
```

この修正は，ハンドラ内からの呼出しを検査し，該当する場合にエラーを返す条件分岐を追加するものである．バイナリ上でも，関数の戻り値に応じて分岐し，`XML_STATUS_ERROR` を返す基本ブロックが追加されていた．提案手法では，この基本ブロックを Add Security Check として検出した．

ただし，表中の脆弱性修正候補の件数は，各関数内で脆弱性修正パターンに一致したすべての基本ブロックを示す．このため，CVE 修正と直接対応しない変更も含まれる．

検証項目 3：同一関数に含まれるその他の変更

表 4 に示すように，`XML_Parse` では，CVE 修正に対応する変更に加えて，CVE に直接対応しない変更も Anchor Node に一致し検出されている．ソースコードを確認したところ，オーバーフロー検査で用いる比較値が変更されており，提案手法ではこの変更を Update Security Check と Add Security Check として検出した．この変更は対象 CVE の PR には含まれていなかったが，入力値に対する検査を強化する変更であり，脆弱性修正と共通する構造的特徴を持つ．

一方，`XML_Parse` には，`EXPAT_FALLTHROUGH` の追加のように，CVE 修正とは関係しない変更も含まれていた．この変更は差分基本ブロックとして変更を検出しているものの，提案手法では脆弱性修正候補として検出しなかった．以上より，提案手法は CVE として明示されていないセキュリティに関わる変更を候補として検出できる一方，すべての変更を脆弱性修正候補として検出するわけではないことを確認した．

4.5　結果に対する考察

本実験では，正解データとした 6 つの外部公開関数すべてについて，CVE 修正に対応する変更を含む基本ブロックを検出できた．一方で，検出結果の大半は Security Check 系であり，ソースコード上の分類と提案手法における分類が一致しない場合も確認された．

© 2026 Information Processing Society of Japan

表 4　脆弱性修正が行われた外部公開関数におけるソースコード分類と提案手法による検出結果

関数名	CVE 対応のソースコード分類	差分基本ブロック数	脆弱性修正候補のパターン内訳
XML_Parse	Add Security Check（PR#1246）	5	Add Security Check: 2，Update Security Check: 1
XML_ParserReset	Update Security Check（PR#1246）	29	Add Security Check: 4，Update Security Check: 6
XML_ParserFree	Update Security Check（PR#1246）	36	Add Security Check: 10，Update Security Check: 7
XML_GetBuffer	Update Security Check（PR#1246）	34	Add Security Check: 5，Update Security Check: 12
XML_ResumeParser	Update Security Check（PR#1267）	2	Add Security Check: 2
XML_ParseBuffer	Update Security Check（PR#1246） Add Security Check（PR#1255）	18	Add Security Check: 11，Update Security Check: 1

この要因は，BinDiff による基本ブロックの対応付けと，コンパイラによる制御フローの展開にある．ソースコード上では既存の条件式を更新する Update Security Check であっても，修正後の条件分岐が追加された基本ブロックとして対応付けられる場合があり，提案手法では Add Security Check として検出される．また，Security Check の Anchor Node は比較命令と条件分岐命令を対象とするため，`for` 文などのループ制御も検出する．したがって，Security Check に一致した基本ブロックの件数は脆弱性修正の件数を直接表すものではなく，検出結果は脆弱性修正候補として解釈する必要がある．

5. ファームウェアを用いたケーススタディ

5.1 実験概要

実験目的： 実機ファームウェアに含まれる strip されたバイナリを対象としても，提案手法がセキュリティに関連し得る変更箇所を抽出可能であるか調査する．

実験対象： ASUS RT-AX86U/RT-AX86S 向け公開ファームウェア `3.0.0.4.388.23285` および `3.0.0.4.388_24231` から抽出した 32 ビット ARM アーキテクチャ向け ELF バイナリ `/usr/sbin/curl` を対象とした．以降，古い方を Primary，新しい方を Secondary と呼ぶ．

実験方法： Secondary のリリースノートには curl の脆弱性修正が記載されている一方で，CVE 番号や修正内容は示されていない．この状況を検証条件として，Primary および Secondary から抽出した `/usr/sbin/curl` に提案手法を適用する．得られた候補のうち，Security Check に分類された基本ブロックを含む関数を Ghidra で逆コンパイルし，検出された命令において条件分岐の追加または更新が検査・拒否処理に関与するかを手作業で確認した．

5.2 実験結果

BinDiff は，更新された関数 `sub_1293C` を一組検出し，その類似度は 0.598 であった．さらに Secondary 側には，処理の一部を切り出した新規関数 `sub_1A7BC` が追加されていた．提案手法による脆弱性修正候補の検出結果を表 5 に示す．BinDiff が検出した 74 個の差分基本ブロックのうち，40 個が少なくとも一つの Anchor Node に一致した．これらから，新たな条件検査に対応し得る候補を 37 箇所，既存の条件検査の更新に対応し得る候補を 8 箇所，および検査処理を新規ヘルパ関数へ移動した候補を 1 件得た．パターンは重複し得るため，カテゴリ別件数の合計は一致基本ブロック数と一致しない．

表 5　ASUS ファームウェア内 `curl` の差分解析結果

項目	件数
差分基本ブロック	74
Anchor Node 一致基本ブロック	40
Anchor Node 非該当基本ブロック	34
Anchor Node 一致箇所	46
Add Security Check 一致基本ブロック（一致箇所）	33 (37)
Update Security Check 一致基本ブロック（一致箇所）	8 (8)
Function Encapsulation 候補	1

Anchor Node に一致した候補を Ghidra で確認したところ，Secondary では呼出し元およびダウンロード URL に対する新たな検査と，条件成立時に `curl` の実行を中止する経路が追加されていた．ASUS のリリースノートは curl の脆弱性修正を示すものの，CVE 番号や修正内容を公開していないため，確認した変更を特定の脆弱性修正と断定することはできない．しかし，提案手法は多数の差分からこのようなセキュリティに関連し得る検査・拒否処理を，追加調査すべき脆弱性修正候補として抽出できた．これは，詳細不明なファームウェア更新に対しても，脆弱性修正候補を提示できることを示している．

6. 議論

本章では，4 章と 5 章の結果を踏まえて，提案手法の有効性，制限事項，および今後の課題について述べる．

6.1 脆弱性修正の特徴を用いたバイナリ差分解析

提案手法では，ソースコード上の脆弱性修正に見られる特徴を基に定義された脆弱性修正パターンを，バイナリ上で観測可能な命令列，制御フロー，および関数呼出し関係へ対応付けた．これにより，ソースコード，CVE 番号，および詳細が不明なバイナリ更新に対しても，脆弱性修正に関連し得る変更候補を抽出することを目指した．

実験では，CVE 関連の修正を含む 6 つの外部公開関数すべてにおいて，対応する変更を含む基本ブロックを検出できた．また，ケーススタディでは，詳細な修正内容が公開されていない strip された `curl` から，脆弱性修正に関

© 2026 Information Processing Society of Japan

連する可能性のある差分候補を抽出できた．以上の結果から，少なくとも調査対象とした事例では，提案手法が，修正内容が未知のファームウェア更新におけるバイナリの差分から脆弱性修正候補を切り分けられる可能性が示せた．

6.2 制限事項

提案手法は，既存研究で整理されたソースコード上の脆弱性修正の特徴をバイナリ上で観測可能な命令列，制御フロー，および関数呼出し関係に対応付けている．したがって，既存の変更カテゴリに含まれない脆弱性修正，またはカテゴリには含まれていても定義したバイナリ上の対応規則に一致しない修正は検出できない可能性がある．また，実行コードを格納するテキスト領域のみを比較対象としている．そのため，`.data`，`.rodata`，`.bss` などのデータ領域における初期値，サイズ，または配置の変更のみで実現される修正は解析できない．

提案手法で利用している BinDiff では，関数の類似度を算出する際に，基本ブロック，制御フローのエッジ，および命令のニーモニックが考慮される．一方，オペランド，即値，およびアドレスが異なる場合でも，ニーモニックが同一であれば同一の命令として扱われることがある．そのため，BinDiff によって差分として識別された基本ブロックのみを解析対象としていると，オペランドや定数のみが変更された局所的な修正を逃す可能性がある．

また，Function カテゴリは API への直接呼出しとシンボル名に依存する．OSS やファームウェアでは，`free` 等の標準ライブラリ関数に相当する処理を内部関数や独自ラッパ関数として実装する場合がある．このような呼出し先は API リストと直接照合できないため，メモリ管理や排他制御に関する変更であっても見逃す可能性がある．

6.3 今後の課題

提案手法では，関数および基本ブロックの差分検出と，差分基本ブロック内の命令の対応付けに BinDiff の出力を利用している．このため，BinDiff が局所的な変更を差分として検出できなかった場合には，当該変更が解析対象から漏れる可能性がある．また，差分が検出された場合でも，バージョン間の命令の対応付けが正確でなければ，命令の追加，削除，および更新を適切に識別できない．今後は，BinDiff による差分検出および命令の対応付けを補完する手法を検討し，局所的な変更の検出と，`ADDED`，`DELETED`，`UPDATED` の判定精度の向上を図る．

7. おわりに

本論文では，修正内容が未知のファームウェア更新から脆弱性修正候補を抽出するため，更新前後の ELF バイナリにおけるアセンブリ命令列および制御フローの差分を，ソースコード上の脆弱性修正に見られる特徴と対応付ける手法を提案した．Expat を用いた実験では，CVE に対応する修正を含む 6 つの外部公開関数すべてについて，対応する変更を含む基本ブロックを検出した．また，ファームウェアに含まれる curl を用いたケーススタディでは，修正内容が公開されていない更新から，セキュリティに関連し得る検査・拒否処理を候補として抽出した．これらの結果から，提案手法が，詳細不明のファームウェア更新における脆弱性修正候補を検出できる可能性を示した．

参考文献

[1] NICTER: NICTER Blog ASUS 製 WiFi ルータの AiCloud 機能の脆弱性を悪用する攻撃に関する注意喚起，https://blog.nicter.jp/2025/04/asus_aicloud.

[2] Dong, C. et al.: LibvDiff: Library Version Difference Guided OSS Version Identification in Binaries, *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pp. 791–802 (online), DOI: 10.1145/3597503.3623336 (2024).

[3] Akiyama, M. et al.: Seeing is not always believing: Insights on IoT manufacturing from firmware composition analysis and vendor survey, *Computers & Security*, Vol. 133, p. 103389 (2023).

[4] Zhang, H. et al.: Precise and accurate patch presence test for binaries, *27th USENIX Security Symposium (USENIX Security 18)*, pp. 887–902 (2018).

[5] Sun, S. et al.: DISPATCH: Unraveling Security Patches from Entangled Code Changes, *34th USENIX Security Symposium (USENIX Security 25)*, pp. 4521–4540 (2025).

[6] Li, F. et al.: A large-scale empirical study of security patches, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2201–2215 (2017).

[7] Zynamics: BinDiff, https://www.zynamics.com/bindiff.html.

[8] National Security Agency: Ghidra version tracking, https://github.com/NationalSecurityAgency/ghidra (2025).

[9] 兼松智也ほか：実行ファイルにおける差分がある関数の特定によるバージョンを変更せずに加えられた脆弱性修正の識別手法，情報処理学会，pp. 1008–1015 (2025).

[10] Jiang, Z. et al.: PDiff: Semantic-based patch presence testing for downstream kernels, *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1149–1163 (2020).

[11] Xu, X. et al.: PatchDiscovery: Patch Presence Test for Identifying Binary Vulnerabilities Based on Key Basic Blocks, *IEEE Transactions on Software Engineering*, Vol. 49, No. 12, pp. 5279–5294 (2023).

[12] Dong, C. et al.: PLocator: Fine-grained Patch Presence Test in Binaries via Patch Code Localization, *ACM Transactions on Software Engineering and Methodology*, Vol. 34, p. 3770079 (online), DOI: 10.1145/3770079 (2025).

[13] Xu, Z. et al.: SPAIN: security patch analysis for binaries towards understanding the pain and pills, *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, IEEE, pp. 462–472 (2017).

[14] He, X. et al.: BinGo: Identifying security patches in binary code with graph representation learning, *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, pp. 1186–1199 (2024).

© 2026 Information Processing Society of Japan

付　　録

A.1　Anchor 候補の検出規則

本手法は, 32 ビット ARM 向け ELF バイナリを対象とし, A32 および T32 の命令を扱う．更新前を Primary，更新後を Secondary とする．対応する命令の opcode と operand が同一の場合を NONCHANGE，異なる場合を UPDATED，Primary のみに存在する場合を DELETED，Secondary のみに存在する場合を ADDED とする．以下では，各カテゴリの Anchor 候補および検出条件を示す.

A.1.1　Sanity Check カテゴリ

ソースコード上の if 文に対応するバイナリ上の Anchor 候補を，表 A·1 の 4 種類に分ける．比較・フラグ設定命令と条件付き分岐の組では，条件付き分岐を Anchor 候補とし，直前の比較・フラグ設定命令を補助条件として用いる．

フラグ定義命令は，CMP，CMN，ADDS，ADCS，SUBS，SBCS，RSBS，RSCS，TST，TEQ，ANDS，EORS，ORRS，ORNS，BICS，MOVS，MVNS，LSLS，LSRS，ASRS，RORS，RRXS，MULS，MLAS，UMULLS，UMLALS，SMULLS，SMLALS とする．

Bcc は，BEQ，BNE，BCS/BHS，BCC/BLO，BMI，BPL，BVS，BVC，BHI，BLS，BGE，BLT，BGT，BLE とする．

A32 条件実行命令は，条件実行可能な基本ニーモニックに条件コードを付加した<base><cc>の形として定義する．基本ニーモニックには，算術・アドレス計算，比較・論理演算，データ移動・シフト，乗算・飽和演算，ビットフィールド・拡張，Load/Store，複数レジスタ転送，排他 Load/Store，分岐・関数呼出し，PSR・コプロセッサ・システム命令を含む．例えば，ADDEQ，MOVNE，LDRGT，STRLS，BLNE，BXEQ を候補とする．条件コードには，表 A·1 に示す AL を除く 14 種類を用いる．ただし，CS と HS，ならびに CC と LO は，それぞれ同一の条件コードを表す別名として扱う．また，条件付き B はフラグ定義命令と Bcc の組として別に検出するため，A32 条件実行命令には含めていない.

以上の命令または命令列を Sanity Check カテゴリの Anchor 候補とし，各命令の差分状態から Add および Update を判定する．Add および Update の判定を表 A·2 に示す．Update では，Primary と Secondary の命令列が互いに対応付けられた基本ブロックに属し，各位置の命令が BinDiff によって対応付けられていることを要求する.

A.1.2　Function カテゴリ

Function カテゴリでは，Anchor 候補として，呼出し先を直接解決できる BL，即値を分岐先とする BLX，および外部 API への末尾呼出しとなる B/B.W を対象とする．objdump から取得したシンボル名から@plt やバージョン表記を除き，表 A·3 の API リストと照合する.

関数呼出しでは，呼出し先シンボルを解決できる直接呼出しのみを検出対象としているため，BLX Rm などの間接呼出し，シンボルを解決できない内部関数，および API リスト外の関数は検出できない．Function Encapsulation についても検出条件を定義しているが，削除処理と追加関数の意味的な同一性までは判定していない.

表 A·1　Sanity Check で対象とする命令形

命令形	定義
フラグ定義＋ Bcc	CMP，CMN，末尾が S の算術・論理・シフト・乗算命令の直後に，Bcc が現れる形
CBZ/CBNZ	比較および分岐を 1 命令で行う T32 命令
A32 条件実行命令	条件実行可能な通常命令に EQ，NE，CS/HS，CC/LO，MI，PL，VS，VC，HI，LS，GE，LT，GT，LE のいずれかを付加した形．(常時実行の AL および Bcc は除く)
T32 IT block	IT，ITT，ITE などと，その後に続く 1–4 命令を一つの命令列とする形

表 A·2　Sanity Check の Add および Update 判定

判定	条件
フラグ定義＋ Bcc	
Add	Secondary の 2 命令がともに ADDED
Update	Primary と Secondary の 2 命令がそれぞれ対応し，フラグ定義命令または Bcc の少なくとも一方が UPDATED
CBZ/CBNZ	
Add	Secondary の命令が ADDED
Update	Primary と Secondary の命令が対応し，opcode または operand の変更により両側で UPDATED
A32 条件実行命令	
Add	Secondary の命令が ADDED
Update	Primary と Secondary の命令が対応し，opcode または operand の変更により両側で UPDATED
T32 IT block	
Add	IT 命令と後続命令がすべて ADDED
Update	Primary と Secondary の同じ長さの IT block を比較し，IT 命令または後続命令の少なくとも 1 命令で opcode または operand が変更された場合

A.1.3　Variable カテゴリ

Variable カテゴリでは，スタック上にゼロ値を書き込む変更，スタック領域のゼロ埋め，およびスタックまたはヒープの確保容量の増加を検出する．各パターンの検出条件を表 A·5 に示す．本手法では，sp または fp を基底レジスタとし，定数オフセットで指定されるメモリ位置をスタック上の固定位置として扱う．格納先の同一性は，Primary と Secondary で基底レジスタおよびオフセットが一致する場合，または格納命令自体が BinDiff によって対応付けられている場合に確認する.

© 2026 Information Processing Society of Japan

表 A·3 Function カテゴリで利用する API リスト

用途	API
メモリ解放	free, cfree, free_sized, free_aligned_sized
C11 lock 取得	mtx_lock, mtx_trylock, mtx_timedlock
mutex 取得	pthread_mutex_lock, pthread_mutex_trylock, pthread_mutex_timedlock, pthread_mutex_clocklock
rwlock 取得	pthread_rwlock_rdlock, pthread_rwlock_tryrdlock, pthread_rwlock_timedrdlock, pthread_rwlock_clockrdlock, pthread_rwlock_wrlock, pthread_rwlock_trywrlock, pthread_rwlock_timedwrlock, pthread_rwlock_clockwrlock
spin lock 取得	pthread_spin_lock, pthread_spin_trylock
semaphore 取得	sem_wait, sem_trywait, sem_timedwait, sem_clockwait

表 A·4 Function カテゴリの検出条件

パターン	検出条件
Add Mem Deallocation Func	Secondary に追加されたメモリ解放 API への直接呼出しを検出
Update Mem Management Func	Primary と Secondary で対応する UPDATED のメモリ解放 API 呼出しを検出
Add Concurrency Control	Secondary に追加された lock 取得 API への直接呼出しを検出
Update Concurrency Control	Primary で削除された lock 取得 API への直接呼出しと Secondary で追加された lock 取得 API への直接呼出しを検出
Function Encapsulation	Primary の削除基本ブロックにあるフラグ定義＋ Bcc または CBZ/CBNZ と，対応する Secondary 関数から追加ヘルパ関数への ADDED/UPDATED 直接呼出しを関連付け

Initialize Variable では，同一基本ブロック内で生成されたゼロ値をスタック上へ格納する形，または memset によってスタック領域をゼロ埋めする形を検出する．

Enlarge Buffer Size では，スタック確保量またはヒープ領域の確保容量が増加した場合を検出する．スタック確保量は，ローカル変数や構造体のメンバ追加によっても変化するため，バッファ拡大の候補として扱う．ヒープ領域については，ARM 32-bit ABI に基づき，表 A·6 に示す引数レジスタから確保容量を求める．容量引数は，同一基本ブロック内の MOV/MOVS，または MOVW と MOVT によって設定された定数に限って取得する．算術命令やメモリ読込みによって決まる値，基本ブロックをまたぐ値，およびレジスタや関数ポインタを介する間接呼出しは検出対象外である．

表 A·5 Variable カテゴリの検出条件

パターン	検出条件
Initialize Variable	対応済み UPDATED 基本ブロックにおいて，Secondary で新たに 0 となった値を同一基本ブロック内の STR，STRB，STRH で既存のスタック上の格納先へ書き込む形を検出．また，r0 が sp/fp 基準アドレス，r1 が 0，r2 が正の定数である memset への直接呼出しにより，スタック領域を新たにゼロ埋めする形も検出
Enlarge Buffer Size (stack)	対応する関数入口基本ブロックの SUB sp, sp, #size を比較し，Secondary の即値が大きい場合を検出
Enlarge Buffer Size (heap)	対応するメモリ確保 API への直接呼出しについて，同一基本ブロック内で設定された定数引数から容量を計算し，Secondary の容量が大きい場合を検出

表 A·6 メモリ確保 API の容量引数

API	引数のレジスタ
malloc(size)	r0
calloc(count, size)	r0×r1
realloc(ptr, size)	r1
reallocarray(ptr, count, size)	r1×r2
aligned_alloc(alignment, size)	r1
posix_memalign(memptr, alignment, size)	r2

A.1.4 Statement カテゴリ

CFG 上で後続基本ブロックを持たず，末尾が BX LR，MOV PC, LR，POP {..., PC}，または LDMIA/LDMFD SP!, {..., PC} などの return に相当する命令で終わる基本ブロックを Return Block Candidate として定義する．ARM 32-bit ABI では返り値が r0 に格納されるため，Return Block Candidate 内を末尾から走査し，r0 を最後に定義する命令を取得する．取得した r0 の最終定義および CFG の変化を用いた各パターンの検出条件を，表 A·7 に示す．

r0 の最終定義は Return Block Candidate 内に存在するものに限って取得し，先行する基本ブロックから到達する定義は追跡できない．また，命令の種類，opcode および operand は比較するが，実行時に r0 へ格納される値までは計算して判定はしていない．

A.1.5 対象範囲

命令集合，条件コード，関数引数レジスタ，返り値レジスタ，および復帰命令は ARM 32-bit ABI と A32/T32 を前提としており，AArch64 や他の命令セットには適用できない．また，解析対象は実行コード領域であるため，.data および .rodata の初期値，ならびに .data，.rodata，.bss のサイズまたは配置だけが変わる修正は検出できない．

© 2026 Information Processing Society of Japan

表 A·7 Statement カテゴリの検出条件

パターン	検出条件
Update Return Value	対応付けられた `UPDATED` 基本ブロックの両側が Return Block Candidate であり，両側に `r0` の最終定義が存在し，opcode または operand が異なる場合を検出
Update Return Type	対応付けられた基本ブロックの両側が Return Block Candidate であり，Primary に `r0` の定義がなく，Secondary に `ADDED` の `r0` 定義がある場合を検出
Enhance Error Handling	Primary では Return Block Candidate であった対応済み基本ブロックが，Secondary では `ADDED` または `UPDATED` の末尾 `B/B.W` を持ち，同一関数内の集約先 Return Block Candidate のみを後続基本ブロックとして持つ場合を検出

© 2026 Information Processing Society of Japan